

Architecture Justification

Purpose and Context

The system is designed to support large-scale analysis of textual data (millions of documents) originating from heterogeneous journalistic and institutional sources. Its primary users are expert analysts who require expressive full-text search, reproducible analytical workflows, and topic-based exploration with response times on the order of seconds. The system is deployed in a controlled, on-premises environment and prioritizes modularity and experimentation over consumer-grade latency.

Architectural Overview

The architecture follows a modular microservices pipeline:

1. **Custom Importer** ingests and normalizes source data.
2. **Elasticsearch** serves as the primary full-text search backend.
3. **NGINX** acts as a reverse proxy in front of the UI and APIs.
4. **Query Handling Service** retrieves cached results from **MongoDB** or executes new searches against Elasticsearch.
5. **Topic Modeling Service (MALLET)** performs analytical enrichment.
6. Query results and analytical outputs are cached for reuse.
7. **Swagger** generates documentation for each microservice's API

This separation of concerns ensures scalability, maintainability, and analytical flexibility.

Justification of Core Design Choices

Elasticsearch as Search Backend

Elasticsearch was chosen due to its ability to efficiently index and query millions of unstructured documents. Its inverted index, ranking capabilities, and aggregation support make it well-suited for exploratory full-text search. Given that query latency of a few seconds is acceptable, Elasticsearch provides an optimal balance between performance and expressiveness for expert users.

MongoDB for Query Caching and Analytical Results

MongoDB is used to store cached queries and derived analytical outputs. As a document-oriented database, it allows flexible schemas that accommodate evolving query definitions, topic modeling parameters, and metadata. This choice supports rapid experimentation and avoids rigid coupling between cached results and the underlying search index structure.

MALLET for Topic Modeling

MALLET was selected as the topic modeling engine due to its status as an academic standard. It provides well-understood, reproducible topic models suitable for research and expert analysis. Decoupling topic modeling from search allows analytical methods to evolve independently of retrieval infrastructure.

Custom Importer

A custom importer enables controlled preprocessing, normalization, and enrichment of heterogeneous data sources. By isolating write-heavy ingestion from read-heavy querying, the system maintains search performance while allowing re-ingestion or re-indexing without impacting downstream services.

Reverse Proxy and Controlled Exposure

NGINX provides a single, secure entry point to the system. Backend services are not directly exposed, and API ports remain closed within the internal network. This design reduces attack surface and simplifies access control(not implemented) in an on-premises deployment.

Swagger API generated docs

Swagger is used to generate the API docs for each one of the microservices which implements a controller. The Swagger microservice retrieves the information from each microservice and generates a webpage containing all the API docs.

This provides a clear separation of concerns as well as a unique place where to consult the API docs.

The API endpoints described in the documentation refer to the internal private network used for inter-service communication. While explicitly exposing internal network addresses in the documentation may appear to conflict with security best practices, this choice is intentional and limited to documentation purposes only. It is adopted to clearly describe and document the system architecture and service interactions, and does not reflect the actual exposure of these endpoints in a production environment.

Custom trainer

The project includes the custom trainer used to train the MALLET topic modeling model.

This choice was made to ensure transparency and reproducibility of the analytical workflow. By providing the trainer as part of the source code, expert users are able to retrain the model using their own datasets and adjust preprocessing steps or modeling parameters according to their specific domain needs. This flexibility is particularly important in research-oriented contexts, where topic quality and interpretability depend strongly on the underlying corpus.

Including the trainer also guarantees consistency between training and inference, as the same data processing logic is reused. Training is treated as an offline, resource-intensive operation and is therefore decoupled from the runtime query pipeline. New models can be generated and deployed without affecting system availability or query performance, supporting experimentation while preserving system stability.

Separated Owilix Dockerfile

The Dockerfile used to build the Owilix image is separated from the main project Dockerfile to preserve modularity and optionality in the data ingestion process. The use of OpenWebIndex data is not a mandatory requirement for the system, as alternative datasets, such as Guardian corpus, are already available and sufficient to populate the pipeline. Additionally, Owilix is not the only possible mechanism for acquiring OWI data; users may choose to directly download datasets from the Open Web Index website, filter and ingest them using the custom scripts and importer.

By isolating the Owilix Dockerfile, the architecture avoids introducing unnecessary dependencies for users who do not require OWI data or who prefer alternative data acquisition workflows. This separation simplifies deployment, reduces build complexity, and aligns with the system's experimentation-oriented design philosophy. Detailed instructions for using Owilix, when chosen, are provided in the project README. This option ensures modularity and prevents a fixed data ingestion strategy.

Architectural Benefits

- **Scalability:** Search and analysis workloads are isolated and can be scaled independently.
 - **Modularity:** Each component can be replaced or extended with minimal impact.
 - **Experimentation:** Flexible schemas and cached analytical results support iterative research workflows. Also alternatives are provided to customize the use.
 - **Fault Isolation:** Failures in analytical services do not compromise ingestion or search.
 - **Reproducibility:** Cached queries and topic models enable consistent analytical results.
-

Deployment Considerations

All services are containerized using Docker to ensure reproducibility and ease of deployment. The system is deployed on-premises to maintain control over data and infrastructure. Resource-intensive components such as Elasticsearch and topic modeling services are isolated to prevent contention. Security is enforced through network segmentation, reverse proxy as only access to the internal network and minimal service exposure.

Conclusion

This architecture intentionally prioritizes analytical rigor, modularity, and controlled scalability. By combining proven search technology, flexible data storage, and academically established modeling tools, the system provides a robust foundation for expert-driven exploration of large textual corpora while remaining adaptable to future research and technical evolution.